



Peer-to-Peer Directory Synchronization with Lightweight Version Control

Arihant Jain

Su Huizhe

Nizar Soufiya

Shengkang Duan

EPITA Graduate School of Computer Science

Advisor

[Put name here]

June 15, 2026

Declaration of Own Work

To be signed in the presence of your Advisor or EPITA staff delegate.

This work has been composed solely by us.

This work has not been accepted in any previous application for a degree or diploma.

The work of which this is a record has been wholly done by us.

All extracts, including those created by generative AI, have been distinguished by quotation marks and the sources of our information have been specifically acknowledged and documented with in-text citations and in the bibliography.

Date: _____

Name: _____

Signature: _____

Name: _____

Signature: _____

Name: _____

Signature: _____

Name: _____

Signature: _____

Acknowledgements

Write your acknowledgements here.

Abstract

The abstract goes here. It should be self-contained and clearly state the problem dealt with by the thesis. Give a brief description of the proposed solution. This section should be more about the problem than the solution. It is recommended to write this section at the same time as the conclusion.

Contents

Declaration of Own Work	1
Acknowledgements	2
Abstract	3
1 Introduction	7
1.1 Motivations	7
1.2 Context of the Project	7
1.3 Objectives	8
1.4 Overview of the Thesis	8
2 Literature Review	10
2.1 The Collaborative File Management Landscape	10
2.2 Critical Analysis of Synchronization Approaches	11
2.2.1 The Ordering Problem	11
2.2.2 Consistency vs. Availability	12
2.2.3 Conflict Resolution: Trade-offs	12
2.2.4 Efficient Data Propagation	13
2.3 Identified Gap	13
2.4 Positioning the Project	14
2.4.1 Key Differentiators	14
2.4.2 Design Principles	14
2.5 Summary	15
3 State of the Art	16
3.1 Zero-Configuration Peer Discovery	16
3.2 Event Ordering and Logical Clocks	17

3.3	Consistency Model and Partition Tolerance	17
3.4	Conflict Resolution and Data Durability	17
3.5	Bandwidth-Efficient Synchronization	18
3.6	Kernel-Level File Watching	18
3.7	Data Integrity and Transport	19
3.8	Summary	19
4	System Architecture	20
4.1	System Architecture Anticipated	20
4.1.1	Components	20
4.1.2	Data Flow	20
4.1.3	Metadata Storage	21
4.1.4	Communication Protocols	21
4.2	System Architecture Updated	22
5	Methodology	23
5.1	Methodology Anticipated	23
5.1.1	Team Structure	23
5.1.2	Development Process	23
5.1.3	Tools	24
5.1.4	Research Design	24
5.1.5	Action Plan	25
5.2	Methodology Updated	25
6	Results	26
6.1	Development Anticipated	26
6.2	Development Updated	27
7	Conclusions	28
	References	30

Appendices	32
A Detailed Comparison Tables	33
B List of Figures	36
C List of Tables	38

Chapter 1

Introduction

1.1 Motivations

University students working in temporary groups need to share files. In practice, most teams send files through group chats, email, or USB drives. This works for a while, but it leads to duplicated files, conflicting versions, and lost work. When two people edit the same document and send it back through the chat, one version gets lost.

Cloud services like Google Drive solve part of this. But they depend on external servers, require account setup, and impose storage limits. Version control systems like Git track changes well but are built for source code and require technical knowledge that not all team members have. Peer-to-peer tools like Syncthing synchronize files without a server but lack version history. If someone saves a broken file, it overwrites everyone else's copy with no undo.

We wanted a tool that fills the space between these options: automatic synchronization on a local network, with version backups built in, and no setup required.

1.2 Context of the Project

This thesis was developed as part of the Action Learning programme at EPITA. Our team has four members: two working on the backend (Go and C++), and two on the frontend and CI/CD (Java and GitHub Actions).

The project grew from our own experience with file sharing problems during previous coursework. We wanted a system where you put files into a folder and they show up on your teammates' machines. Edit a file and the change spreads automatically. Accidentally overwrite something and a backup is there.

1.3 Objectives

The project has three main objectives:

1. Design a serverless peer-to-peer synchronization system that discovers peers automatically on a local network using mDNS/DNS-SD, synchronizes directories through direct TCP connections, and requires no manual configuration.
2. Provide version safety by creating local backups automatically before any synchronized overwrite, using Lamport clock timestamps and a Last-Write-Wins conflict resolution policy.
3. Validate the design through two testable hypotheses:

H_1 : Sub-second average synchronization latency on a standard WLAN without cloud infrastructure.

H_2 : Sufficient consistency and rollback recovery for 3 to 5 peers using single logical timestamps with LWW, without the overhead of vector clocks or CRDTs.

1.4 Overview of the Thesis

Chapter 2 reviews the literature on distributed synchronization and peer-to-peer systems. It analyses the trade-offs between existing approaches and identifies the gap that this project addresses.

Chapter 3 presents the state of the art: the specific research and technical decisions that shaped the architecture.

Chapter 4 describes the system architecture, including the Go/C++ component split and the communication protocols used.

Chapter 5 covers the methodology, including the action plan, development process, and research design.

Chapter 6 presents the expected development results and, after Action Learning Week, the actual outcomes.

Chapter 7 contains the conclusions and outlines future work.

The appendices include detailed comparison tables and supplementary material.

Chapter 2

Literature Review

This chapter reviews the research that informed our project. It focuses on what we learned from the literature, how it changed our thinking, and where open questions remain. The emphasis is on analysis and lessons learned, not on describing what each tool does.

Our team split the research by topic. Arihant and Huizhe focused on distributed systems theory (consistency models, replication, synchronization algorithms). Nizar and Shengkang studied existing collaborative tools and local networking technologies. We met weekly to present findings and discuss how they applied to our project.

2.1 The Collaborative File Management Landscape

Collaborative file management tools fall into four broad categories. Each one does some things well and other things poorly.

Centralized platforms like Google Drive (Google, 2026), Dropbox (Dropbox, 2026), and Nextcloud are easy to use and synchronize files well. But they depend entirely on external servers. When the server is down or the internet is unavailable, collaboration stops. Their version history is shallow: enough to undo a recent mistake, but not enough to track how a project changed over weeks.

Version control systems like Git (Chacon & Straub, 2014) and SVN provide detailed revision histories, branching, and conflict resolution. But they were designed for source code. Their workflow (staging, committing, pushing, pulling, resolving merge conflicts) assumes technical knowledge that not every member of a student team has.

Peer-to-peer synchronization tools like Syncthing (Syncthing Community, 2026) and Resilio Sync (Resilio, Inc., 2026) remove the server requirement and synchronize files in the

background. They handle any file type. But their version management is basic. Syncthing has file versioning, but it is a snapshot-based add-on, not a core feature. If someone saves a broken file, it immediately replaces the file on all connected devices.

Local sharing technologies like AirDrop and Nearby Share offer simple one-time file transfers. There is no ongoing synchronization, no history, and no shared workspace.

The main takeaway from this survey is that no existing tool combines automatic peer-to-peer synchronization with built-in version safety. The full comparison is in Appendix A.

2.2 Critical Analysis of Synchronization Approaches

The academic literature on distributed synchronization raises several problems that directly affected our design choices.

2.2.1 The Ordering Problem

In a distributed system, things happen on different machines at different times. Without a shared clock, deciding which event came first is not straightforward. Lamport (1978) solved this with logical clocks: each process keeps a counter, increments it on every event, and attaches it to messages. If event A caused event B , then A 's counter is lower than B 's.

This is useful but incomplete. Lamport clocks give a partial ordering. They cannot tell you whether two events happened at the same time with no connection between them. Fidge (1988) proposed vector clocks to solve this: each node keeps a separate counter for every participant. Vector clocks detect concurrent events, but they grow with the number of peers and require all participants to be known in advance.

For our system (3 to 5 peers), we decided vector clocks add more complexity than value. A single Lamport counter per file, with a tiebreaker rule, is enough. This is a practical choice, not a perfect one. If we needed to scale beyond a handful of peers, we would need to revisit it.

2.2.2 Consistency vs. Availability

The CAP theorem, proven by Gilbert and Lynch (2002), says that a distributed system experiencing network problems must choose between consistency (all nodes see the same data) and availability (all nodes can read and write). You cannot have both during a partition.

For our use case, the choice is straightforward. Student laptops go to sleep, disconnect from Wi-Fi, and leave the building. Blocking edits until everyone is online would make the tool useless. So we chose availability: peers edit freely when offline, and differences are resolved when they reconnect.

Vogels (2009) calls this “eventually consistent” and argues it is the right model when temporary staleness is acceptable. A teammate seeing a file that is a few seconds old is fine. Losing a file because the system blocked an edit during a disconnection is not.

2.2.3 Conflict Resolution: Trade-offs

When two peers change the same file at the same time, the system must decide what to do. The literature shows a range of approaches.

Operational transformation (Sun & Ellis, 1998) tracks individual editing operations and transforms them to preserve intent. Google Docs uses this. But it requires character-level logging, which does not work for binary files like images.

CRDTs (Shapiro, Preguiça, Baquero, & Zawirski, 2011) define data structures whose merge operations always converge, regardless of the order updates are applied. Shapiro et al. proved this mathematically. It is a strong result, but implementing CRDTs for arbitrary file metadata requires careful design and testing.

Last-Write-Wins (LWW) keeps whichever version has the highest timestamp. One version is lost. The Bayou system (Petersen, Spreitzer, Terry, Theimer, & Demers, 1995) showed that LWW works when combined with backups: before overwriting, save the old version. Terry et al. (1994) added “session guarantees” so users always see their own writes, even when other updates have not arrived yet.

We chose LWW with local backups, following Bayou’s approach. The overwritten file is

saved before being replaced. This gives us simple conflict resolution with the safety of full recoverability. Our metadata schema is designed so we can add CRDT-based merging later, using the approach described by Shapiro et al. (2011).

Saito and Shapiro's (2005) survey of optimistic replication confirms that this pattern (local updates with deferred reconciliation and rollback) is standard for systems with weak connectivity.

2.2.4 Efficient Data Propagation

Even on a local network, transferring whole files every time is wasteful. Tridgell and Mackerras (1996) showed that rolling checksums let two machines find which blocks of a file changed and transfer only the differences. This is how rsync works.

Demers et al. (1987) studied a different problem: how to spread updates across a group of peers reliably. Their "epidemic algorithms" model update propagation like a contagion. Each peer that learns of an update passes it to its neighbours, who pass it to theirs, until everyone has it.

Our initial design uses whole-file transfers for simplicity. The architecture can accommodate both optimizations later: block-level transfers in the C++ daemon, and gossip-based propagation in the Go coordinator.

2.3 Identified Gap

Comparing existing tools across five dimensions (membership management, synchronization, history management, file support, and filesystem transparency) shows a clear pattern. Each tool category does well in some areas and poorly in others:

- Centralized platforms: strong synchronization and usability, weak history management.
- Version control systems: strong history management, weak synchronization automation and usability for non-technical users.

- P2P synchronization tools: strong synchronization and usability, weak history management.
- Local sharing: strong usability, no synchronization or history.

The gap is not the absence of any single feature. It is the absence of a tool that does well across all five dimensions at once. Each existing tool was designed for a different purpose (enterprise documents, software development, device mirroring, casual sharing), so none of them covers our use case.

See Appendix A for the full breakdown.

2.4 Positioning the Project

2.4.1 Key Differentiators

Our system differs from existing tools in four ways:

- **LAN-only operation.** Discovery uses mDNS/DNS-SD on the local network. No external servers, no accounts, no internet needed.
- **No configuration.** Peers are found automatically. No device IDs to exchange, no invitations to approve, no repositories to create.
- **Built-in version backups.** Every overwrite saves the old file first. This is a core feature, not an optional setting.
- **Split architecture.** Go handles networking; C++ handles file watching and hashing. This keeps resource use low on student laptops.

2.4.2 Design Principles

1. **Simplicity.** The shared folder should behave like a normal directory. No staging, no commits, no merge conflicts.

2. **Durability.** Automation must not lose data. Every overwrite keeps the old version. Users can always go back.
3. **Performance.** The tool runs in the background. It should not drain battery or slow down the machine.

2.5 Summary

Three things from the literature shaped this project the most:

First, the CAP theorem means we must choose availability over consistency. Students cannot be expected to coordinate edits or wait for the network. The system has to work offline and reconcile later.

Second, conflict resolution is a trade-off. For a small number of peers editing mixed file types, LWW with backups gives the best balance of simplicity and safety. CRDTs are better in theory but add complexity our timeline does not allow. Our schema supports adding them later.

Third, no existing tool targets the design space we are working in. This is not because the space is unimportant, but because existing tools were built for different users. Our contribution is putting together known techniques (mDNS, Lamport clocks, LWW, TCP) into a package built for small-team, short-term collaboration.

Chapter 3

State of the Art

This chapter describes the state of the art from the research we conducted for this project. It covers the technical decisions we made for technology, architecture, and synchronization design, with citations for each choice. Each section maps a research finding to a component in our system and explains why we chose one approach over the alternatives.

3.1 Zero-Configuration Peer Discovery

We chose Multicast DNS (mDNS) and DNS-Based Service Discovery (DNS-SD) (Cheshire & Krochmal, 2013b, 2013a) for peer discovery. These protocols let devices on a local network advertise and find services using standard IP multicast, without a central DNS server or manual setup.

Our Go discovery component broadcasts service records under the `_p2psync._tcp` domain and scans the subnet for peer addresses. Users do not need to exchange IP addresses, device IDs, or invitation codes. This removes a source of friction we observed in Syncthing’s setup process (Syncthing Community, 2026).

One limitation: mDNS relies on multicast, which some enterprise networks filter. For our target use case (students on campus Wi-Fi), this is acceptable. Supporting more restricted networks would require a relay server, which would reintroduce the server dependency we are trying to avoid.

3.2 Event Ordering and Logical Clocks

Lamport (1978) showed that logical clocks can order events in a distributed system without synchronized physical clocks. Each process keeps a counter, increments it on events, and attaches it to messages. If event A caused event B , A 's timestamp is lower than B 's.

We attach a single Lamport counter to each file's metadata. When a peer changes a file, it increments the counter and sends the update. Receiving peers compare timestamps to decide if the incoming version is newer.

The known limitation is that Lamport clocks cannot detect concurrent edits (two peers changing the same file at the same time with no connection between them). Fidge (1988) proposed vector clocks to solve this, but they grow with the number of peers and add protocol complexity. For 3 to 5 peers, we judged the overhead unnecessary. We use Lamport ordering with a Last-Write-Wins tiebreaker and local backups before every overwrite, following the approach documented in the Bayou system (Petersen et al., 1995).

3.3 Consistency Model and Partition Tolerance

The CAP theorem (Gilbert & Lynch, 2002) proves that during a network partition, a distributed system must choose between consistency and availability. Student laptops go to sleep, disconnect, and leave the room. Blocking edits until all peers are reachable would make the tool useless.

We chose availability: peers edit freely while offline, and differences are reconciled when connectivity returns. Vogels (2009) calls this "eventually consistent" and argues it fits systems where temporary staleness is acceptable. Our backup mechanism ensures that even if reconciliation overwrites a file, no data is permanently lost.

3.4 Conflict Resolution and Data Durability

We chose LWW for the initial release, following Bayou (Petersen et al., 1995). The key idea from Bayou is that simple resolution policies work when paired with backups. Before any

overwrite, our C++ daemon saves the existing file into a local `.versions/` directory. LWW resolves the conflict automatically, and users can recover the overwritten version if needed.

Saito and Shapiro’s (2005) survey of optimistic replication confirms that this pattern (optimistic updates with rollback support) is standard for weakly connected systems. Their taxonomy classifies our approach as “state transfer” (sending full file states) rather than “operation transfer” (sending edit operations). State transfer is simpler to implement but uses more bandwidth.

We designed our SQLite metadata schema to support state-based CRDTs in a future version, using the register-based approach from Shapiro et al. (2011).

3.5 Bandwidth-Efficient Synchronization

Tridgell and Mackerras (1996) showed that rolling checksums let two machines identify which blocks of a file changed and transfer only the differences. This is how rsync works.

Our initial design transfers whole files for simplicity. When a file changes, the entire file goes over a direct TCP connection (Eddy, 2022). This is fine for small files but wasteful for large ones. Block-level delta synchronization is planned as a future optimization. The C++ daemon is designed with an abstraction layer that can accommodate partial transfers without changing the Go coordination layer.

3.6 Kernel-Level File Watching

Detecting file changes efficiently matters for a tool that runs continuously in the background. Polling wastes CPU and adds latency. Modern operating systems provide kernel-level event APIs: `inotify` on Linux (Kerrisk, 2024) and `FSEvents` on macOS (Apple, 2007). These push notifications when files are created, changed, or deleted.

Our C++ daemon registers watchers on the synchronized directory. Events are grouped and forwarded to the Go coordination engine through IPC. This gives near-instant detection with minimal CPU use.

3.7 Data Integrity and Transport

File integrity verification uses SHA-256 hashing (National Institute of Standards and Technology, 2015), performed by the C++ daemon with memory-mapped file I/O. Each file's hash is stored in the local metadata database and exchanged during synchronization to detect changes without comparing file contents byte by byte.

Network transport uses TCP (Eddy, 2022) for reliable, ordered delivery. The Go daemon manages TCP connection pools with discovered peers and hands over sockets to the C++ daemon for direct-to-disk file streaming.

3.8 Summary

Table 3.1 summarizes how each research finding maps to a component in our system.

Table 3.1: Research-to-Implementation Mapping

Research Finding	Source	System Component
Zero-config discovery	RFC 6762/6763	Go mDNS broadcaster
Causal event ordering	Lamport (1978)	Per-file Lamport counters
Partition tolerance	CAP theorem (2002)	Offline-first availability
Conflict resolution	Bayou (1995)	LWW + local backups
Optimistic replication	Saito & Shapiro (2005)	State-transfer architecture
Delta synchronization	rsync (1996)	Planned block-level sync
File change detection	inotify / FSEvents	C++ kernel watchers
Data integrity	SHA-256 / TCP	C++ hashing + Go TCP pools

Chapter 4

System Architecture

This chapter describes the design adopted to achieve the objectives stated in the Introduction. The system splits work between two cooperating processes: a Go networking component and a C++ storage component, connected by IPC.

4.1 System Architecture Anticipated

4.1.1 Components

The system has three main components:

1. **Go Networking Daemon.** Handles peer discovery through mDNS/DNS-SD, manages TCP connections with peers, coordinates synchronization state, and runs an IPC server for communication with the storage daemon and the frontend.
2. **C++ Storage Daemon.** Handles local filesystem work: file watching (inotify/FSEvents), SHA-256 hashing with memory-mapped I/O, version backups before overwrites, and writing incoming files to disk.
3. **Java Frontend.** A graphical interface for monitoring sync status, browsing version history, and managing shared folders. It talks to the Go daemon through IPC.

4.1.2 Data Flow

A typical synchronization cycle works as follows:

1. The C++ watcher detects a file change and computes the new SHA-256 hash.

2. It sends a metadata update (file path, hash, Lamport timestamp) to the Go daemon through IPC.
3. The Go daemon increments the file's Lamport counter and sends the update to connected peers.
4. Receiving peers compare the incoming timestamp against their local metadata.
5. If the incoming version is newer, the Go daemon requests a file transfer from the sender.
6. The file is sent over TCP directly to the C++ daemon on the receiving machine.
7. Before writing the new file, the C++ daemon saves the existing version in the `.versions/` directory.
8. The new file is written and the local metadata database is updated.

4.1.3 Metadata Storage

Each peer keeps a local SQLite database with:

- File path (relative to the sync root)
- SHA-256 content hash
- Lamport timestamp
- Last modification time (for display only)
- File size in bytes

The schema is designed so it can be extended to support CRDTs later without breaking the existing protocol.

4.1.4 Communication Protocols

- **Discovery:** mDNS multicast on the local subnet (port 5353), advertising the `_p2psync._tcp` service.

- **Coordination:** Custom protocol over TCP for metadata exchange (file lists, version comparisons, transfer requests).
- **File Transfer:** Direct TCP streaming between peers. The Go daemon sets up the connection and passes the socket to C++ for I/O.
- **IPC:** Unix domain sockets between the Go and C++ daemons on the same machine.

4.2 System Architecture Updated

This section will be completed at the end of Action Learning Week. It will document any changes to the architecture that came up during implementation, including modifications to the component structure, protocol changes, and lessons learned.

Chapter 5

Methodology

This chapter discusses the methodology used in the project, the stages of implementation, and the research design. It covers the participants, the tools used, the development procedure, and how results will be evaluated. This is where our action plan fits.

5.1 Methodology Anticipated

5.1.1 Team Structure

The team has four members, split by area:

- **Arihant Jain:** Backend (Go). Peer discovery, synchronization coordination, IPC server, TCP networking.
- **Su Huizhe:** Backend (C++). File watching, SHA-256 hashing, file versioning, disk I/O.
- **Nizar Soufiya:** Frontend (Java). User interface and interaction design.
- **Shengkang Duan:** Frontend (Java) and CI/CD. Build automation, testing pipeline, GitHub Actions.

5.1.2 Development Process

We used an iterative, prototype-based approach rather than planning everything upfront:

1. **Phase 1: Research and Design.** Literature review, architecture design, and protocol specification. This produced the literature review, architecture diagrams, and the interface contracts between Go and C++.

2. **Phase 2: Core Prototyping.** Independent development of the Go networking daemon and C++ storage daemon. Each component was built and tested against mock interfaces before integration.
3. **Phase 3: Integration.** Connecting Go and C++ through IPC, testing end-to-end synchronization on a local network, and connecting the Java frontend.
4. **Phase 4: Evaluation.** Testing the system against the hypotheses from the Introduction: measuring synchronization latency (H_1) and evaluating consistency and recovery (H_2).

5.1.3 Tools

- **Languages:** Go 1.21+, C++17, Java 17
- **Build:** Go modules, CMake (C++), Gradle (Java)
- **Version control:** Git with GitHub
- **CI/CD:** GitHub Actions
- **Testing:** Go testing package, Google Test (C++), JUnit (Java)
- **Test environment:** Multiple laptops on the same WLAN
- **Documentation:** L^AT_EX

5.1.4 Research Design

We formulated two hypotheses and will test them through controlled experiments:

- H_1 (**Latency**): Measure sync latency across 3 to 5 peers on a standard WLAN. Success means sub-second average latency for files under 10 MB.
- H_2 (**Consistency**): Simulate concurrent edits and network disconnections. Success means:
(a) no data loss from conflict resolution, (b) all peers reach the same file state within 30 seconds of reconnection, and (c) overwritten versions can be recovered from the backup directory.

5.1.5 Action Plan

Table 5.1: Action Plan Timeline

Period	Work
Pre-ALW	Literature review, architecture design, component prototypes
ALW Day 1–2	IPC integration between Go and C++ daemons
ALW Day 3–4	End-to-end sync testing, frontend integration
ALW Day 5	Hypothesis testing, writing up results

5.2 Methodology Updated

This section will be completed at the end of Action Learning Week. It will document the actual process followed, including any changes from the plan, problems encountered, and adjustments made.

Chapter 6

Results

This chapter covers the expected results from the project and, after Action Learning Week, the actual outcomes.

6.1 Development Anticipated

Based on the architecture and methodology described above, we expect the following:

1. **Go Networking Daemon** that can:

- Discover peers on the local network through mDNS/DNS-SD.
- Maintain TCP connections with discovered peers.
- Exchange file metadata (paths, hashes, Lamport timestamps) to detect changes.
- Request and coordinate file transfers.
- Communicate with the local C++ daemon through Unix domain sockets.

2. **C++ Storage Daemon** that can:

- Watch the synchronized directory for changes using kernel-level APIs (inotify on Linux, FSEvents on macOS).
- Compute SHA-256 hashes of changed files using memory-mapped I/O.
- Save existing files to a `.versions/` directory before overwriting.
- Write incoming file data directly to disk from a TCP socket.

3. **Java Frontend** that shows:

- Current sync status (connected peers, progress).

- File version history with the option to restore old versions.
- Workspace settings (sync directory, peer list).

4. Hypothesis Testing:

H_1 : Sync latency measurements across 3 to 5 peers on campus WLAN, targeting sub-second averages for files under 10 MB.

H_2 : Concurrent edit tests showing no permanent data loss, convergence within 30 seconds, and recoverability from backups.

6.2 Development Updated

This section will be completed at the end of Action Learning Week. It will describe the differences between what was planned and what was actually built, including features that were added, changed, or deferred.

Maximum one-half page.

Chapter 7

Conclusions

This thesis addressed the problem of file synchronization for small collaborative teams that need something simpler than Git but safer than basic file mirroring.

We identified a gap in existing tools. Centralized services need external infrastructure and accounts. Version control systems require technical knowledge and manual workflows. Peer-to-peer sync tools lack version safety. No existing tool combines automatic local-network synchronization with built-in version backups and zero configuration.

Our system fills this gap with a two-process architecture. The Go networking daemon handles peer discovery (mDNS/DNS-SD), synchronization coordination (Lamport clocks with LWW conflict resolution), and TCP file transfer. The C++ storage daemon handles file watching, hashing, backups, and disk I/O. Together, they provide a shared directory that behaves like a normal folder while synchronizing across peers on the same network.

The design makes trade-offs that fit our target use case:

- Lamport clocks instead of vector clocks: simpler but cannot detect truly concurrent edits.
- Last-Write-Wins instead of CRDTs: simpler but one version is discarded (with backup).
- Whole-file transfers instead of delta sync: simpler but uses more bandwidth.

These choices work for 3 to 5 peers on a local network. Scaling further would require revisiting them.

Future work includes:

- Block-level delta synchronization using the rsync rolling-checksum algorithm to reduce bandwidth for large files.
- CRDT-based metadata merging to allow conflict-free convergence without LWW.

- Cross-subnet discovery for peers on different networks.
- Mobile platform support, where filesystem access and background execution work differently.

The project shows that a simple, serverless synchronization tool is feasible by combining well-understood protocols (mDNS, TCP, SHA-256) with a practical conflict resolution approach (LWW with backups).

References

- Apple. (2007). *File system events programming guide*. https://developer.apple.com/library/archive/documentation/Darwin/Conceptual/FSEvents_ProgGuide/Introduction/Introduction.html. (Accessed: 2026-05-28)
- Chacon, S., & Straub, B. (2014). *Pro git* (2nd ed.). Apress.
- Cheshire, S., & Krochmal, M. (2013a). *DNS-based service discovery*. RFC 6763. Retrieved from <https://www.rfc-editor.org/rfc/rfc6763> (Accessed: 2026-05-28)
- Cheshire, S., & Krochmal, M. (2013b). *Multicast DNS*. RFC 6762. Retrieved from <https://www.rfc-editor.org/rfc/rfc6762> (Accessed: 2026-05-28)
- Demers, A., Greene, D., Hauser, C., Irish, W., Larson, J., Shenker, S., ... Terry, D. (1987). Epidemic algorithms for replicated database maintenance. In *Proceedings of the sixth annual acm symposium on principles of distributed computing* (pp. 1–12). doi: 10.1145/41840.41841
- Dropbox. (2026). *Dropbox help center*. <https://help.dropbox.com/>. (Accessed: 2026-05-28)
- Eddy, W. M. (2022). *Transmission control protocol (TCP)*. RFC 9293. Retrieved from <https://www.rfc-editor.org/rfc/rfc9293> (Accessed: 2026-05-28)
- Fidge, C. J. (1988). Timestamps in message-passing systems that preserve the partial ordering. In *Proceedings of the 11th australian computer science conference* (pp. 56–66).
- Gilbert, S., & Lynch, N. (2002). Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59.
- Google. (2026). *Google drive help: Use drive for desktop*. <https://support.google.com/drive/>. (Accessed: 2026-05-28)
- Kerrisk, M. (2024). *inotify(7) — linux manual page*. <https://man7.org/linux/man-pages/man7/inotify.7.html>. (Accessed: 2026-05-28)
- Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 558–565.

- National Institute of Standards and Technology. (2015). *FIPS PUB 180-4: Secure hash standard (SHS)*. Retrieved from <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf> (Accessed: 2026-05-28)
- Petersen, K., Spreitzer, M., Terry, D., Theimer, M., & Demers, A. (1995). Managing update conflicts in Bayou, a weakly connected replicated storage system. In *Proceedings of the 15th acm symposium on operating systems principles* (pp. 172–182). doi: 10.1145/224056.224070
- Resilio, Inc. (2026). *Resilio sync documentation*. <https://help.resilio.com/hc/en-us>. (Accessed: 2026-05-28)
- Saito, Y., & Shapiro, M. (2005). Optimistic replication. *ACM Computing Surveys*, 37(1), 42–81. doi: 10.1145/1057977.1057980
- Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). A comprehensive study of convergent and commutative replicated data types. *INRIA Research Report*(RR-7506).
- Sun, C., & Ellis, C. (1998). Operational transformation in real-time group editors: Issues, algorithms, and achievements. In *Proceedings of the acm conference on computer supported cooperative work* (pp. 59–68). doi: 10.1145/289444.289469
- Syncthing Community. (2026). *Syncthing documentation*. <https://docs.syncthing.net/>. (Accessed: 2026-05-28)
- Terry, D. B., Demers, A. J., Petersen, K., Spreitzer, M. J., Theimer, M. M., & Welch, B. B. (1994). Session guarantees for weakly consistent replicated data. In *Proceedings of the third international conference on parallel and distributed information systems* (pp. 140–149). doi: 10.1109/PDIS.1994.331722
- Tridgell, A., & Mackerras, P. (1996). The rsync algorithm. In *Proceedings of the unix annual technical conference*.
- Vogels, W. (2009). Eventually consistent. *Communications of the ACM*, 52(1), 40–44. doi: 10.1145/1435417.1435432

Appendices

Appendix A

Detailed Comparison Tables

The following tables provide a detailed, dimension-by-dimension comparison of existing collaborative file management approaches. They supplement the analysis presented in Chapter 2 by providing granular ratings for individual systems within each category.

The qualitative ratings used are defined in Table A.1.

Table A.1: Qualitative Rating Scale

Rating	Meaning
High	Strong support for the capability
Partial	Partial support with notable limitations
Limited	Basic or restricted support
None	Capability not provided

Table A.2: Comparison: Membership Management

Category	System	Peer Discovery	Access Control	Group Formation
Centralized	<i>Google Drive</i>	Limited	High	High
	<i>OneDrive</i>	Limited	High	High
	<i>Dropbox</i>	Limited	High	High
	<i>Nextcloud</i>	Limited	High	Partial
Version Control	<i>Git</i>	None	None	Limited
	<i>SVN</i>	None	High	Limited
	<i>Perforce</i>	None	High	Limited
P2P Sync	<i>Syncthing</i>	High	Partial	Partial
	<i>Resilio Sync</i>	Partial	Partial	Partial
Local Sharing	<i>AirDrop</i>	High	None	Limited
	<i>Nearby</i>	High	None	Limited
	<i>Share</i>			

Table A.3: Comparison: Synchronization Model

Category	System	Automatic Sync	Offline Operation	Multi-Device Sync
Centralized	<i>Google Drive</i>	High	Limited	High
	<i>OneDrive</i>	High	Limited	High
	<i>Dropbox</i>	High	Limited	High
	<i>Nextcloud</i>	High	Partial	High
Version Control	<i>Git</i>	None	High	Partial
	<i>SVN</i>	None	Limited	Partial
	<i>Perforce</i>	None	Limited	Partial
P2P Sync	<i>Syncthing</i>	High	High	High
	<i>Resilio Sync</i>	High	High	High
Local Sharing	<i>AirDrop</i>	None	None	None
	<i>Nearby Share</i>	None	None	None

Table A.4: Comparison: History Management

Category	System	Version History	Rollback	Conflict Resolution
Centralized	<i>Google Drive</i>	Limited	Limited	Limited
	<i>OneDrive</i>	Limited	Limited	Limited
	<i>Dropbox</i>	Limited	Limited	Limited
	<i>Nextcloud</i>	Partial	Partial	Limited
Version Control	<i>Git</i>	High	High	High
	<i>SVN</i>	High	High	High
	<i>Perforce</i>	High	High	High
P2P Sync	<i>Syncthing</i>	Partial	Partial	Limited
	<i>Resilio Sync</i>	Partial	Partial	Limited
Local Sharing	<i>AirDrop</i>	None	None	None
	<i>Nearby Share</i>	None	None	None

Table A.5: Comparison: File Support

Category	System	Heterogeneous Files	Large Files
Centralized	<i>Google Drive</i>	High	Partial
	<i>OneDrive</i>	High	Partial
	<i>Dropbox</i>	High	Partial
	<i>Nextcloud</i>	High	High
Version Control	<i>Git</i>	Partial	Limited
	<i>SVN</i>	Partial	Partial
	<i>Perforce</i>	High	High
P2P Sync	<i>Syncthing</i>	High	High
	<i>Resilio Sync</i>	High	High
Local Sharing	<i>AirDrop</i>	High	High
	<i>Nearby</i>	High	High
	<i>Share</i>		

Table A.6: Comparison: Filesystem Transparency and Usability

Category	System	Filesystem Integration	Ease of Use
Centralized	<i>Google Drive</i>	High	High
	<i>OneDrive</i>	High	High
	<i>Dropbox</i>	High	High
	<i>Nextcloud</i>	High	Partial
Version Control	<i>Git</i>	Limited	Limited
	<i>SVN</i>	Limited	Limited
	<i>Perforce</i>	Limited	Limited
P2P Sync	<i>Syncthing</i>	High	High
	<i>Resilio Sync</i>	High	High
Local Sharing	<i>AirDrop</i>	High	High
	<i>Nearby</i>	High	High
	<i>Share</i>		

Table A.7: Gap Summary Across Evaluation Dimensions

Approach	Membership Management	Sync	History Management	File Support	Filesystem Transparency
Centralized Platforms	High	High	Limited	High	High
Version Control Systems	Partial	Limited	High	Partial	Limited
P2P Sync Systems	Partial	High	Limited	High	High
Local Sharing	Limited	None	None	High	High
Desired Solution	High	High	High	High	High

Appendix B

List of Figures

List of Figures

Appendix C

List of Tables

List of Tables

3.1	Research-to-Implementation Mapping	19
5.1	Action Plan Timeline	25
A.1	Qualitative Rating Scale	33
A.2	Comparison: Membership Management	33
A.3	Comparison: Synchronization Model	34
A.4	Comparison: History Management	34
A.5	Comparison: File Support	35
A.6	Comparison: Filesystem Transparency and Usability	35
A.7	Gap Summary Across Evaluation Dimensions	35